- 1. Software Development
  - Software methodology
    - 定义：structured process used to create software
    - 包括：planning、analysis、design、coding、testing、deployment、maintenance
    - 例子：Waterfall、Agile
  - SDLC 七阶段
    - 1 Requirements：What does the customer need?
    - 2 Analysis：Understand the problem
    - 3 Design：Plan the solution
    - 4 Coding：Write the program
    - 5 Testing：Check whether it works
    - 6 Deployment：Release/install for users
    - 7 Maintenance：Fix bugs + add features
  - Agile vs Waterfall
    - Agile
      - iterative（迭代） cycles
      - requirements can change
      - frequent feedback
      - 每个 cycle 可发布新版本
    - Waterfall
      - sequential order
      - fixed requirements
      - 每个阶段通常依次完成一次
  - Programming paradigm type
    - 编程范式
    - Imperative命令式：一步步告诉电脑 how to do it
      - Procedural
      - Object-Oriented
    - Declarative声明式：描述 what result is wanted
      - Functional
        - Haskell
      - Logical
        - Prolog
  - Procedural vs OOP
    - Procedural：以 procedures/functions 为中心
    - OOP：以 classes/objects 为中心
      - OOP commonly uses this step-by-step approach through: Objects, Methods, Variables, Assignment statement, Control structures
    - 课程题：传统 procedural program may use shared global variables

- 2. Requirements & Use Case
  - Requirements analysis
    - refine requirements
    - transform requirements into user stories
    - 回答：what need or do / who / conditions,limiations
    - 不等于 software design
    - 过程: group answers, use-case diagram, user-stories table, convert user-stories
  - User story
    - As a [user], I want to [action], so that [outcome].
  - Requirements validation：4C
    - Compliance：是否符合规则?
    - Correctness：是否真是客户需要的?
    - Completeness：是否遗漏?
    - Consistency：是否互相矛盾?
  - Documentation testing/reviewing
    - 目的：ensure the correctness of requirements
    - 查找：missing、unclear、contradictory、uncorrect、untestable requirements
  - Actor
    - 定义：person / organisation / device / external system interacting with the system
    - 可以是：Customer、Manager、Printer、Cash Register
    - 不一定是人
  - Use case
    - 定义：a goal an actor wants to achieve
    - 通常用动词：Log in、Withdraw、Make Deposit
    - Login 是 use case，不是 actor
  - Use Case Diagram
    - 回答：What should the system do?
    - 不展示：Java/Python code、SQL、API implementation
    - 元素
      - Actor：小人
      - Use case：椭圆
      - System boundary：系统范围方框
      - Association：实线
      - <
  - Sequence Diagram
    - 显示 participants、messages、time order、how changes
    - 最具体分类：Interaction Diagram
    - 广义分类：Behaviour Diagram

- 3. Class、Object、OOP 与 OOD
  - Class
    - blueprint / template used to create objects
    - 包含 attributes(data) + methods(behaviors)
  - Object
    - instance created from a class
    - Class = recipe(食谱)；Object = cake
    - Objects exist at runtime and store their own data.
  - Attribute
    - data stored inside a class/object
    - 例子：balance、name、customerId
  - Method / operation
    - what an object can do
    - 例子：deposit()、withdraw()、login()
  - 辨认方法
    - noun → potential class/attribute
    - verb/action → method
    - Deposit 是 action；通常是 deposit() method
  - OOP 四原则：AEIP
    - Abstraction
      - hide complexity
      - show only what is needed
        - **Data abstraction** → Hide the variable itself, just show the value *(e.g., you see your bank balance, not how it's stored in code)*
        - **Function abstraction** → Hide the code, just use the function *(e.g., use withdraw() without seeing how it works inside)*
    - Encapsulation
      - bundle data + related methods in one class
      - control access to data
      - 例：private balance + public getBalance()
    - Inheritance
      - child inherits/reuses parent attributes and methods
      - is-a relationship
      - 例：Manager is an Employee
      - 多个 subclasses 可以继承同一个 superclass
    - Polymorphism
      - same method name, different behaviours
      - 例：不同 objects 的 make_sound() 结果不同
  - 易混淆
    - bundle functions with data → Encapsulation，不是 Inheritance
    - sibling subclasses 共享 parent type，但不是同一个 concrete class
    - multiple subclasses → one superclass：可以
    - one subclass → multiple superclasses：取决于编程语言
  - Procedure vs Function（课程口径）
    - Procedure：performs an action；通常不返回值　pay()、deposit()、printReceipt()
    - Function：usually returns a value　getBalance()、calculateInterest()
    - 课程题：pay is a procedure
  - OOD
    - 1. Find the classes (from real use cases)
    - 2. List their attributes (nouns) and methods (verbs)
    - 3. Show how they relate
    - 4. Draw a UML class diagram

- 4. UML Class Diagram
  - Class box 三部分
    - Top：Class name
    - Middle：Attributes
    - Bottom：Methods / operations
  - Visibility
    - + public
    - - private
    - # protected
  - Method notation
    - + methodName(parameter: type): returnType
    - + deposit(amount: double): void
    - void = returns no value
  - Relationships
    - Association
      - works with / uses
      - Customer - Account
    - Aggregation
      - has-a；weak ownership
      - 空心菱形放在 whole 一端
      - part can exist without the whole
    - Generalization / Inheritance
      - is-a
      - 空心三角形指向 parent/superclass
      - Manager → Employee
  - Multiplicity
    - 1 = exactly one
    - 0..1 = zero or one
    - 0..* = zero or many
    - 1..* = one or many
    - 1..5 = one to five
  - 例题
    - Bank 1 - 1..5 Branch
      - one Bank has between one and five Branch objects
  - Use Case → Class Diagram
    - nouns/actors → potential classes
    - verbs → methods
    - data → attributes
    - identify relationships
    - add multiplicity
    - review + justify

- 5. Exercise 1
  - Q1 Software methodology
    - process used to create programs
  - Q2 NOT an Agile characteristic
    - sequential SDLC with fixed requirements
  - Q3 Imperative paradigm
    - Object-Oriented
  - Q4 Procedural programming
    - intended answer：functions may access shared global variables
    - 更准确定义：program organised into procedures/functions
  - Q5 NOT correct about OOP
    - class data is available to all system functions
  - Q6 Requirements analysis
    - refines/transforms requirements into user stories
  - Q7 Documentation testing
    - ensure correctness of requirements
  - Q8 NOT requirements-analysis step
    - create a class diagram
  - Q9 Cannot be an actor
    - Login
  - Q10 Sequence Diagram
    - Interaction Diagram
- 6. Exercise 2
  - Q1 Class
    - template code used to create objects
  - Q2 Object
    - created by executing/instantiating class code
  - Q3 NOT an OOP principle
    - Parallel
  - Q4 "Inheritance bundles functions with data"
    - False：这是 Encapsulation
  - Q5 "Encapsulation bundles data and related functions"
    - True
  - Q6 "Multiple classes can inherit the same class"
    - True
  - Q7 Cashier 和 Printer 都继承 Machine
    - True：都属于 Machine parent type
    - 但 Cashier 和 Printer 不是同一个 concrete class
  - Q8 NOT an OOD step
    - identify system actors
  - Q9 pay operation
    - pay is a procedure（课程答案）
  - Q10 Bank 1 - Branch 1..5
    - True：至少 1 个，最多 5 个 Branch objects

7. 考场陷阱词

找错误或不属于的选项	not / cannot / incorrect
选最准确的定义，而不是一个可能特征	best describes
Sequence Diagram → Interaction Diagram	most specific
绝对词需要特别检查	all / only / always
表示可能，不等于必须	may / can

8. 最后 30 秒速记

- Class = blueprint
- Object = instance
- Attribute = data
- Method = behaviour
- Abstraction = hide complexity
- Encapsulation = bundle + control access
- Inheritance = is-a
- Polymorphism = same method, different behaviour
- Association = works with
- Aggregation = has-a
- Generalization = is-a
- 0..* = zero or many
- 1..* = one or many
- Sequence Diagram = Interaction Diagram